

```

import 'package:flutter/material.dart';
import 'package:geolocator/geolocator.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'package:foodfinder/views/restaurant_list_screen.dart';
import 'package:foodfinder/views/add_restaurant_screen.dart';
import 'package:foodfinder/services/geolocation_service.dart';
import 'package:foodfinder/services/notification_service.dart';

class MainScreen extends StatefulWidget {
  const MainScreen({super.key});

  @override
  State<MainScreen> createState() => _MainScreenState();
}

class _MainScreenState extends State<MainScreen> {
  String locationMessage = "Location not available";
  final GeolocationService _geoService = GeolocationService();
  final NotificationService _notificationService = NotificationService();
  bool isMuted = false;

  @override
  void initState() {
    super.initState();
    _initNotifications();
    _loadMuteSetting();
  }

  Future<void> _initNotifications() async {
    await _notificationService.init();
    await _notificationService.requestPermission();
  }

  Future<void> _loadMuteSetting() async {
    final prefs = await SharedPreferences.getInstance();
    setState(() {
      isMuted = prefs.getBool('notificationsMuted') ?? false;
    });
  }

  Future<void> _toggleMute() async {
    final prefs = await SharedPreferences.getInstance();
    setState(() {
      isMuted = !isMuted;
      prefs.setBool('notificationsMuted', isMuted);
    });
  }
}

```

```

ScaffoldMessenger.of(context).showSnackBar(
  SnackBar(
    content: Text(
      isMuted ? 'Notifications muted' : 'Notifications unmuted',
    ),
    duration: const Duration(seconds: 2),
  ),
);
}

Future<void> getLocation() async {
  try {
    Position position = await _geoService.getCurrentPosition();
    setState(() {
      locationMessage =
        "Latitude: ${position.latitude}, Longitude:
${position.longitude}";
    });
  } catch (e) {
    setState(() {
      locationMessage = "Error: $e";
    });
  }
}

void _gotoRestaurantList() {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const RestaurantListScreen()),
  );
}

void _gotoAddRestaurant() {
  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => const AddRestaurantScreen()),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: GestureDetector(
        onTap: () {
          showDialog(
            context: context,
            builder: (context) => AlertDialog(

```

```

        content: Image.asset('assets/images/easter_egg.png'),
        actions: [
          TextButton(
            onPressed: () => Navigator.of(context).pop(),
            child: const Text('Close'),
          ),
        ],
      ),
    );
  },
  child: const Text('Food Finder'),
),
actions: [
  IconButton(
    icon: Icon(
      isMuted ? Icons.notifications_off : Icons.notifications_active,
    ),
    tooltip: isMuted ? 'Unmute Notifications' : 'Mute Notifications',
    onPressed: _toggleMute,
  ),
],
),
body: LayoutBuilder(
  builder: (context, constraints) {
    final isSmall = constraints.maxWidth < 600;
    final isMedium = constraints.maxWidth < 1200;

    final children = [
      Card(
        margin: const EdgeInsets.all(12),
        child: Padding(
          padding: const EdgeInsets.all(16),
          child: Column(
            children: [
              Text(locationMessage),
              const SizedBox(height: 8),
              ElevatedButton.icon(
                icon: const Icon(Icons.my_location),
                label: const Text('Get Current Location'),
                onPressed: getLocation,
              ),
            ],
          ),
        ),
      ),
      ElevatedButton.icon(
        icon: const Icon(Icons.restaurant_menu),
        label: const Text('Restaurant List'),
      ),
    ],
  ),
),

```

```

        onPressed: _gotoRestaurantList,
      ),
      ElevatedButton.icon(
        icon: const Icon(Icons.add_business),
        label: const Text('Add Restaurant'),
        onPressed: _gotoAddRestaurant,
      ),
    ],
  );

  if (isSmall) {
    return Center(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(16),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: children,
        ),
      ),
    );
  } else if (isMedium) {
    return Center(
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
          Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: children.sublist(0, 2),
          ),
          Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: children.sublist(2),
          ),
        ],
      ),
    );
  } else {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Wrap(
          spacing: 24,
          runSpacing: 24,
          alignment: WrapAlignment.center,
          children: children,
        ),
      ),
    );
  }
}

```

```
    },  
    ),  
    );  
}  
}
```